

# Лабораторная работа №4

---

## Хранение настроек и данных профиля в SharedPreferences

О чем лабораторная?

В приложении уже есть экраны и навигация, но после закрытия приложения введенные данные пропадают. В этой работе добавим экран профиля, сохраним его данные в **SharedPreferences** и сделаем так, чтобы OnBoard показывался только при первом запуске.

Главное, что нужно понять

- **SharedPreferences** подходит для небольших настроек и простых данных;
  - данные хранятся как пары "ключ - значение" и переживают перезапуск приложения;
  - запись выполняется через `edit()` и `apply()`;
  - настройки профиля и флаг прохождения OnBoard лучше хранить отдельно.
- 

### Часть 1. Цель работы

Научиться сохранять данные приложения между запусками с помощью **SharedPreferences**: создать экран профиля, сохранять введенные пользователем данные, загружать их при следующем открытии приложения, а также использовать **SharedPreferences** для хранения флагов - например, показывать экраны приветствия (**OnBoard**) только при первом запуске.

### Почему данные исчезают

В предыдущих лабораторных мы создавали экраны и навигацию. Но есть проблема: все данные, которые вводит пользователь, хранятся только в оперативной памяти. Закройте приложение - и все, что было введено, исчезнет.

Для небольших данных - настроек приложения и профиля пользователя - в Android есть простое решение: **SharedPreferences**. Это встроенное хранилище, которое сохраняет данные в виде пар "ключ-значение" в файл на устройстве. При следующем запуске приложения данные можно прочитать обратно.

**SharedPreferences** подходит для:

- настроек приложения (тема, уведомления, язык)
- профиля пользователя (имя, email, описание)
- флагов состояния (например, "приветственный экран уже показан")
- любых небольших объектов простых типов: **String**, **int**, **boolean**, **float**

Для больших и структурированных данных **SharedPreferences** не подходит - позже мы будем использовать базу данных.

---

### Часть 2. Практическая часть

## Шаг 1. Создайте экран профиля

Создайте новую активити - `ProfileActivity`. Добавьте ее в `AndroidManifest.xml`, если Android Studio не сделала это автоматически.

В разметке `activity_profile.xml` разместите:

- `EditText` для имени пользователя (`et_username`)
- `EditText` для email (`et_email`)
- `EditText` для описания о себе (`et_about_me`) - многострочный, с ограничением в 500 символов
- кнопку "Сохранить" (`btn_save`)

Ограничение длины для `et_about_me` задайте в XML:

```
android:maxLength="500"
```

## Шаг 2. Создайте класс-помощник

Работа с `SharedPreferences` напрямую из активити засоряет код. Вынесите ее в отдельный класс - `ProfilePreferences`.

```
public class ProfilePreferences {

    private static final String PREFS_NAME = "profile_prefs";
    private static final String KEY_USERNAME = "username";
    private static final String KEY_EMAIL = "email";
    private static final String KEY_ABOUT_ME = "about_me";

    private final SharedPreferences prefs;

    public ProfilePreferences(Context context) {
        prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    }

    public void saveProfile(String username, String email, String aboutMe) {
        prefs.edit()
            .putString(KEY_USERNAME, username)
            .putString(KEY_EMAIL, email)
            .putString(KEY_ABOUT_ME, aboutMe)
            .apply();
    }

    public String getUsername() {
        return prefs.getString(KEY_USERNAME, "");
    }

    public String getEmail() {
        return prefs.getString(KEY_EMAIL, "");
    }

    public String getAboutMe() {
```

```

        return prefs.getString(KEY_ABOUT_ME, "");
    }
}

```

Разберем, что здесь происходит:

- `getSharedPreferences(PREFS_NAME, MODE_PRIVATE)` - открывает файл настроек с именем `profile_prefs`. Если файла еще нет, он будет создан при первой записи
- `prefs.edit()` - начинаем изменение данных. Изменения применяются только после `apply()` или `commit()`
- `apply()` - сохраняет изменения асинхронно, не блокируя интерфейс. Используйте его в большинстве случаев
- `getString(KEY, "")` - читает строку по ключу. Второй аргумент - значение по умолчанию, если данные еще не сохранены

### Шаг 3. Сохранение данных

В `ProfileActivity` добавьте обработчик кнопки "Сохранить":

```

public class ProfileActivity extends AppCompatActivity {

    private EditText etUsername;
    private EditText etEmail;
    private EditText etAboutMe;
    private ProfilePreferences profilePrefs;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_profile);

        etUsername = findViewById(R.id.et_username);
        etEmail = findViewById(R.id.et_email);
        etAboutMe = findViewById(R.id.et_about_me);
        Button btnSave = findViewById(R.id.btn_save);

        profilePrefs = new ProfilePreferences(this);

        btnSave.setOnClickListener(v -> {
            profilePrefs.saveProfile(
                etUsername.getText().toString(),
                etEmail.getText().toString(),
                etAboutMe.getText().toString()
            );
            Toast.makeText(this, "Профиль сохранен", Toast.LENGTH_SHORT).show();
        });
    }
}

```

### Шаг 4. Загрузка данных при открытии

Чтобы данные появлялись в полях при каждом открытии экрана, загрузите их в `onCreate` после инициализации полей:

```
etUsername.setText(profilePrefs.getUsername());
etEmail.setText(profilePrefs.getEmail());
etAboutMe.setText(profilePrefs.getAboutMe());
```

## Шаг 5. Показ OnBoard только при первом запуске

В лабораторной №3 вы создали экраны приветствия (`OnBoard`). Сейчас они показываются при каждом запуске приложения. Это неправильно: приветствие нужно показать только один раз - при первом запуске.

Для этого сохраним флаг "OnBoard пройден" в SharedPreferences. Создайте отдельный класс для настроек приложения - `AppPreferences`:

```
package com.example.avito;

import android.content.Context;
import android.content.SharedPreferences;

public class AppPreferences {

    private static final String PREFS_NAME = "app_prefs";
    private static final String KEY_ONBOARDING_COMPLETED = "onboarding_completed";

    private final SharedPreferences prefs;

    public AppPreferences(Context context) {
        prefs = context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE);
    }

    public boolean isOnboardingCompleted() {
        return prefs.getBoolean(KEY_ONBOARDING_COMPLETED, false);
    }

    public void setOnboardingCompleted(boolean completed) {
        prefs.edit()
            .putBoolean(KEY_ONBOARDING_COMPLETED, completed)
            .apply();
    }
}
```

Теперь измените `MainActivity` (или активити, с которой начинается приложение), чтобы она проверяла флаг:

```
public class MainActivity extends AppCompatActivity {

    @Override
```

```

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    AppPreferences appPrefs = new AppPreferences(this);

    if (!appPrefs.isOnboardingCompleted()) {
        startActivity(new Intent(this, OnBoardActivity.class));
        finish();
        return;
    }

    setContentView(R.layout.activity_main);
}
}

```

А в **OnBoardActivity** после прохождения всех экранов сохраните флаг:

```

AppPreferences appPrefs = new AppPreferences(this);
appPrefs.setOnboardingCompleted(true);
startActivity(new Intent(this, MainActivity.class));
finish();

```

## Итоговый результат

- Экран профиля с тремя текстовыми полями и кнопкой "Сохранить"
- Класс **ProfilePreferences** для работы с SharedPreferences
- Данные сохраняются при нажатии "Сохранить" и загружаются при открытии экрана
- Данные сохраняются после полного перезапуска приложения
- Экраны OnBoard показываются только при первом запуске - при последующих запусках приложение сразу открывает главный экран

## Дополнительно

1. Добавьте на экран профиля переключатель **Switch** с текстом "Получать уведомления". Сохраняйте его состояние в SharedPreferences как **boolean** и загружайте при открытии экрана.
2. Добавьте кнопку "Очистить профиль", которая удаляет все сохраненные данные. Для этого в **ProfilePreferences** добавьте метод:

```

public void clearProfile() {
    prefs.edit().clear().apply();
}

```

3. Добавьте простую валидацию полей перед сохранением:

- имя пользователя - от 3 до 32 символов, только латиница
- email - используйте **Patterns.EMAIL\_ADDRESS.matcher(...).matches()**
- описание о себе - не более 500 символов

Если какое-то поле не проходит проверку, покажите `Toast` с понятным сообщением и не сохраняйте данные. Валидацию выполняйте в `ProfileActivity` перед вызовом `saveProfile`